PH 3120- Computational Physics Laboratory- I

CPL 102-Numerical Differentiation and Integration

WEEK-03

Name – Tharaka Lenadora
Reg. num – 2022s19441
Index num- s16820

Q1) First Derivatives

i.

a) Taylor series for an arbitrary function ((x)) at $x = a$.

$$f(x) = \sum_{n=0}^{\infty} \frac{f^n(a)}{n!}(x-a)^n$$

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots$$

For first order numerical differentiation

$$f(x) = \sum_{n=1}^{\infty} \frac{f^1(a)}{1!}(x-a)^1$$

$$f(x) = f(a) + f'(a)(x-a)$$

b) 1. Forward difference:

Consider the Taylor series expansion of $f(x+h)$ around x:

$$f(x+h) = f(x) + f'(x)(h) + \frac{f''(x)}{2!}(h)^2 + \frac{f'''(x)}{3!}(h)^3 + \cdots$$

$$f(x+h) - f(x) = f'(x)(h) + \frac{f''(x)}{2!}(h)^2 + \frac{f'''(x)}{3!}(h)^3 + \cdots$$

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{f''(x)}{2!}h + \frac{(x)}{3!}(h)^2 + \cdots$$

For a small h, the higher order terms become negligible.

So,

$$\frac{f(x+h) - f(x)}{h} = f'(x) + O(h)$$

$$\boldsymbol{f'(x) = \frac{f(x+h) - f(x)}{h}}$$

2. Backward difference:

Consider the Taylor series expansion of $f(x-h)$ around x:

$$f(x-h) = f(x) + f'(x)(-h) + \frac{f''(x)}{2!}(-h)^2 + \frac{f'''(x)}{3!}(-h)^3 + \cdots$$

$$f(x) - f(x-h) = f'(x)(h) - \frac{f''(x)}{2!}(h)^2 + \frac{f'''(x)}{3!}(h)^3 + \cdots$$

$$\frac{f(x) - f(x-h)}{h} = f'(x) - \frac{f''(x)}{2!}h + \frac{f'''(x)}{3!}(h)^2 + \cdots$$

For a small h, the higher order terms become negligible.

So,

$$\frac{f(x) - f(x-h)}{h} = f'(x) + O(h)$$

$$\boldsymbol{f'(x) = \frac{f(x) - f(x-h)}{h}}$$

3.Central Difference:

The Taylor series expansion of $f(x+h)$ around x is:

$$f(x+h) = f(x) + f'(x)(h) + \frac{f''(x)}{2!}(h)^2 + \frac{f'''(x)}{3!}(h)^3 + \cdots (1)$$

The Taylor series expansion of $f(x-h)$ around x is:

$$f(x-h) = f(x) - f'(x)(h) + \frac{f''(x)}{2!}(h)^2 - \frac{f'''(x)}{3!}(h)^3 + \cdots (2)$$

$(1) - (2)$:

$$f(x+h) - f(x-h) = f(x) + f'(x)(h) + \frac{f''(x)}{2!}(h)^2 + \frac{f'''(x)}{3!}(h)^3 - \left(f(x) - f'(x)(h) + \frac{f''(x)}{2!}(h)^2 - \frac{f'''(x)}{3!}(h)^3\right) + O(h) - O(h)$$

$$f(x+h) - f(x-h) = 2f'(x)(h) + \frac{2f'''(x)}{3!}(h)^3 + O(h)$$

So,

$$\frac{f(x+h) - f(x-h)}{h} = f'(x) + O(h)$$

$$\boldsymbol{f'(x) = \frac{f(x+h) - f(x-h)}{h}}$$

c) The truncation error in a Taylor series is the difference between the actual value of a function and the value given by the Taylor approximation because we only use a limited number of term.

d) In Forward Difference Method,

$$f'(x) = \frac{f(x+h) - f(x)}{h}$$

But in the Model,

$$f'(x) = \frac{f(x+h)-f(x)}{h} - \left(\frac{f''(x)}{2!}h + \frac{f'''(x)}{3!}(h)^2 + \cdots\right)$$

So, the Truncation error is given by,

$$TE = \left(\frac{f''(x)}{2!}h + \frac{f'''(x)}{3!}(h)^2 + \cdots\right)$$

In First order,

$$TE = \frac{f''(x)}{2!}h$$

As well as in Backward Difference Method,

$$TE = \left(\frac{f''(x)}{2!}h - \frac{f'''(x)}{3!}(h)^2 + \cdots\right)$$

In First order,

$$TE = -\frac{f''(x)}{2!}h$$

In Central Difference Method,

$$TE = \left(\frac{2f'''(x)}{3!}(h)^3 + \frac{2f'''''(x)}{5!}(h)^5 + \cdots\right)$$

In First order,

$$TE = 0$$

Consider about these 3 methods, Central difference Method has lowest truncation error.

e)

Model	advantages	disadvantages	application
forward	• Simple and easy to implement • Requires only current and next point	• Less accurate (error is proportional to h) • Not symmetric	• Good when data is available only from current and future points • Often used in real-time systems or initial value problems
backward	• Simple • Uses current and previous data	• Same accuracy as forward (error is proportional to h) • Not symmetric	• Useful when only past data is available • Often used in time-dependent simulations (e.g., after collecting data)
Central	• More accurate (error is proportional to h^2) • Symmetric, reduces error	• Requires data on both sides of the point • Can't use at boundaries (like start or end of data)	• Best for interior points • Common in scientific computing and simulations where accuracy matters

f) The choice of step size (h) in numerical differentiation is very important because it directly affects both accuracy and stability of the derivative. When we use too big h values it occurs high truncation error. When we use too small h it makes round off error and unstable. There's a best (optimal) step size where the total error is lowest.

Method	Effect of step size
Forward	Accuracy improves as $h \to 0$, but round-off error increases if h is too small
Backward	Same as forward difference

Central	More accurate (error $\propto h^2$) $\rightarrow$ works better even with a slightly larger h.

g)

Truncation Error: Happens because we use an approximation instead of the exact derivative. Gets smaller when step size h is smaller.

Round-off Error: Caused by computer's limited precision. Gets worse when h is too small.

How error vary with step size and smoothness:

With Step Size h:

- Big hhh → more truncation error.
- Small hhh → more round-off error.
- Best accuracy is at an "optimal" step size.

With Smoothness of Function:

- Smooth functions (like curves) → less error.
- Rough or sharp functions → more error.

ii.

[ ]:
```
# Q1 II)
```

19]:
```
h=2.3
x=5.43

def f(x):
    return x**2-1

def num_diff(f, x: float, h: float, method: str):
    if method == 'forward':
        return (f(x + h) - f(x)) / h
    else:
        if method == 'backward':
            return (f(x) - f(x - h)) / h
        else:
            if method == 'central':
                return (f(x + h) - f(x - h)) / (2 * h)
            else :
                print ("Invalid method. Choose 'forward', 'backward', or 'central'.")

print("Forward Difference:", num_diff(f, x, h, 'forward'))
print("Backward Difference:",num_diff(f, x, h, 'backward'))
print("Central Difference:", num_diff(f, x, h, 'central'))
```

```
Forward Difference: 13.159999999999998
Backward Difference: 8.559999999999999
Central Difference: 10.859999999999998
```

iii.

[35]:
```
import numpy as np
arr=np.array([0.5,0.2,0.1,0.05,0.01,0.001])
Ex=3*x**2+2
print("Expected value of sloution:",Ex)
print(" ")

def f(x):
    return x**3+2*x-1
for i in range(0,5):
    h=arr[i]
    x=2
    def num_diff(f, x: float, h: float, method: str):
        if method == 'forward':
            return (f(x + h) - f(x)) / h
        else:
            if method == 'backward':
                return (f(x) - f(x - h)) / h
            else:
                if method == 'central':
                    return (f(x + h) - f(x - h)) / (2 * h)
                else :
                    print ("Invalid method. Choose 'forward', 'backward', or 'central'.")
    fw =num_diff(f, x, h, 'forward')
    bk =num_diff(f, x, h, 'backward')
    cn = num_diff(f, x, h, 'central')
    print("Forward Difference when h=",h,"is",fw)
    print("Backward Difference when h=",h,"is",bk)
    print("Central Difference when h=",h,"is",cn)
    print(" ")
    Ef=((Ex-fw)/fw)*100
    Eb=((Ex-bk)/bk)*100
```

```python
    print("Central Difference when h=",h,"is",cn)
    print(" ")
    Ef=((Ex-fw)/fw)*100
    Eb=((Ex-bk)/bk)*100
    Ec=((Ex-cn)/cn)*100
    print("Forward Difference Error prentage when h=",h,"is",Ef,"%")
    print("Backward Difference Error prentage when h=",h,"is",Eb,"%")
    print("Central Difference Error prentage when h=",h,"is",Ec,"%")
    print(" ")
```

```
Expected value of sloution: 14

Forward Difference when h= 0.5 is 17.25
Backward Difference when h= 0.5 is 11.25
Central Difference when h= 0.5 is 14.25

Forward Difference Error prentage when h= 0.5 is -18.84057971014493 %
Backward Difference Error prentage when h= 0.5 is 24.444444444444443 %
Central Difference Error prentage when h= 0.5 is -1.7543859649122806 %

Forward Difference when h= 0.2 is 15.240000000000018
Backward Difference when h= 0.2 is 12.839999999999998
Central Difference when h= 0.2 is 14.040000000000008

Forward Difference Error prentage when h= 0.2 is -8.136482939632653 %
Backward Difference Error prentage when h= 0.2 is 9.034267912772602 %
Central Difference Error prentage when h= 0.2 is -0.2849002849003419 %

Forward Difference when h= 0.1 is 14.61000000000002
Backward Difference when h= 0.1 is 13.41000000000001
Central Difference when h= 0.1 is 14.010000000000016
```


```
Forward Difference when h= 0.2 is 15.240000000000018
Backward Difference when h= 0.2 is 12.839999999999998
Central Difference when h= 0.2 is 14.040000000000008

Forward Difference Error prentage when h= 0.2 is -8.136482939632653 %
Backward Difference Error prentage when h= 0.2 is 9.034267912772602 %
Central Difference Error prentage when h= 0.2 is -0.2849002849003419 %

Forward Difference when h= 0.1 is 14.61000000000002
Backward Difference when h= 0.1 is 13.41000000000001
Central Difference when h= 0.1 is 14.010000000000016
```


```
Forward Difference Error prentage when h= 0.1 is -4.175222450376591 %
Backward Difference Error prentage when h= 0.1 is 4.399701715137873 %
Central Difference Error prentage when h= 0.1 is -0.07137758743765713 %

Forward Difference when h= 0.05 is 14.302499999999938
Backward Difference when h= 0.05 is 13.702500000000022
Central Difference when h= 0.05 is 14.00249999999998

Forward Difference Error prentage when h= 0.05 is -2.1150148575419636 %
Backward Difference Error prentage when h= 0.05 is 2.1711366538951116 %
Central Difference Error prentage when h= 0.05 is -0.017853954650812114 %

Forward Difference when h= 0.01 is 14.060099999999665
Backward Difference when h= 0.01 is 13.940099999999944
Central Difference when h= 0.01 is 14.000099999999804

Forward Difference Error prentage when h= 0.01 is -0.42745072936654765 %
Backward Difference Error prentage when h= 0.01 is 0.42969562628715746 %
Central Difference Error prentage when h= 0.01 is -0.0007142806108839871 %
```

2. Second Order Derivatives

i.

a) Consider the Taylor series expansion of $f(x+h)$ around x:
f(x+h) = f(x) + f'(x)h + f''(x)(h^2)/2! + f'''(x)(h^3)/3! +……

Consider the Taylor series expansion of $f(x-h)$ around x:
f(x-h) = f(x) - f'(x)h + f''(x)(h^2)/2! - f'''(x)(h^3)/3! +……
by adding above two expressions,
f(x+h) + f(x-h) = 2f(x) + 2f''(x)(h^2)/2! + 2f''''(x)(h^4)/4! +……
f''(x) = [f(x+h) - 2f(x) + f(x-h)]/h^2 - f''''(x)(h^2)/12 +……

For a sufficiently small h, the higher-order terms become negligible,

Then the expression for 2nd order derivative based on the central difference method is,
f''(x) = [f(x+h) - 2f(x) + f(x-h)]/h^2

b) f''(x) = [f(x+h) - 2f(x) + f(x-h)]/h^2 - f''''(x)(h^2)/12 +……
The truncation error in the central difference formula for approximating the second derivative,

TE =(Exact value)−(Approximate value)

TE = - f''''(x)(h^2)/12 +……
The order of accuracy is 2 which is indicated by $O(h$^2)

c) No, the standard central difference method cannot be directly applied if the step size is non-uniform as the derivation assumes equal spacing, f(x+h) and f(x−h) must be at the same distance h from x.

d) The central difference method assumes the function is smooth (continuous and differentiable) and Well-behaved with equal spacing.

If the function has discontinuities derivatives may not exists.So the approximation becomes invalid.If the function has sharp changes it may lead to large truncation errors if h is too large as large derivatives equire very small step sizes for accuracy.And also the problem with functions with higher frequency components is numerical differentiation tends to amplify noise and oscillations, reducing accuracy.

ii.

Q2

[5]:
```
h = 2.3
x = 5.43


def f(x):
    return x**2 - 1

def central_diff(f, x: float, h: float, order: int):
    if order == 1:
        return (f(x + h) - f(x - h)) / (2 * h)
    elif order == 2:
        return (f(x + h) - 2 * f(x) + f(x - h)) / (h ** 2)
    else:
        return None

print("Central Difference 1st derivative:", central_diff(f, x, h, 1))
print("Central Difference 2nd derivative:", central_diff(f, x, h, 2))
```

```
Central Difference 1st derivative: 10.859999999999998
Central Difference 2nd derivative: 1.9999999999999996
```

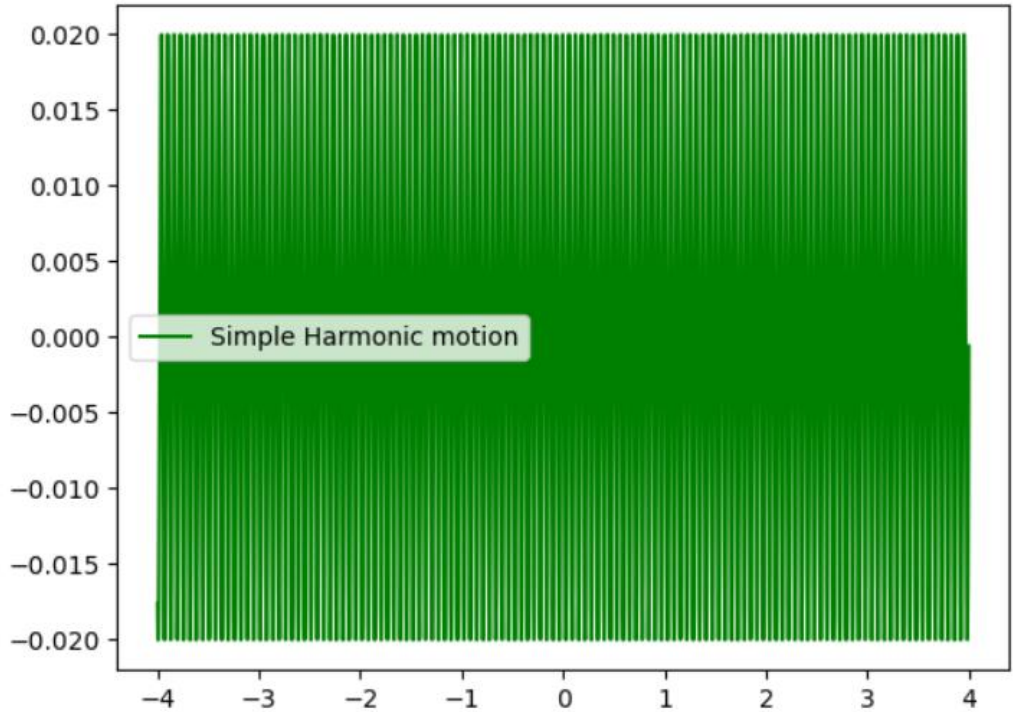
iii.

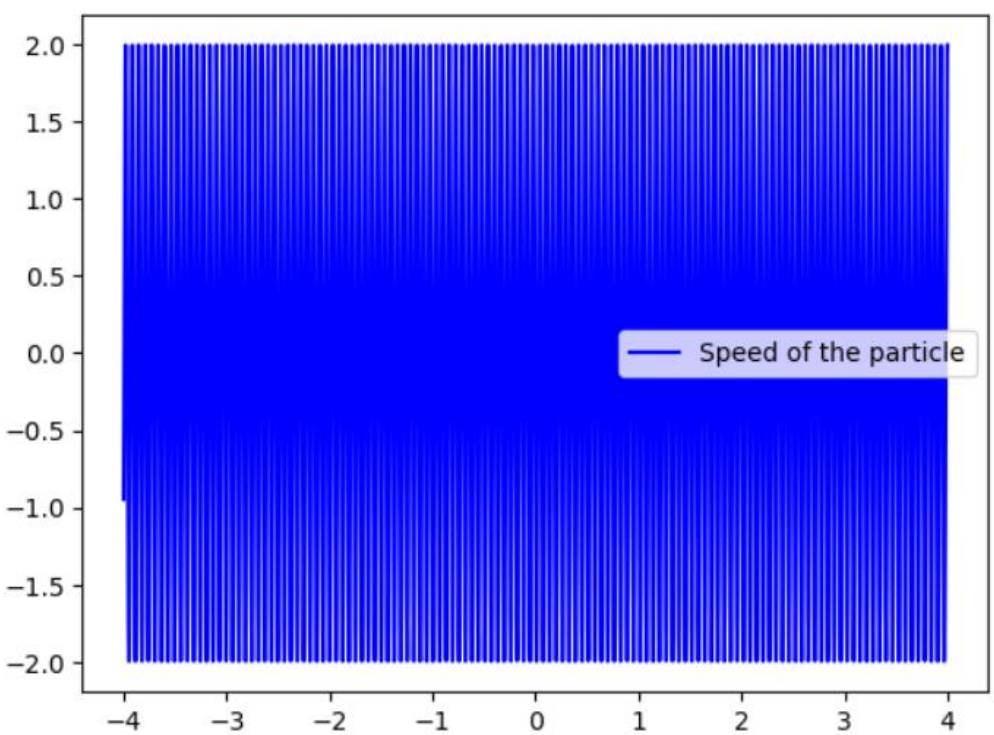
[ ]:
```
# Q iii)
```

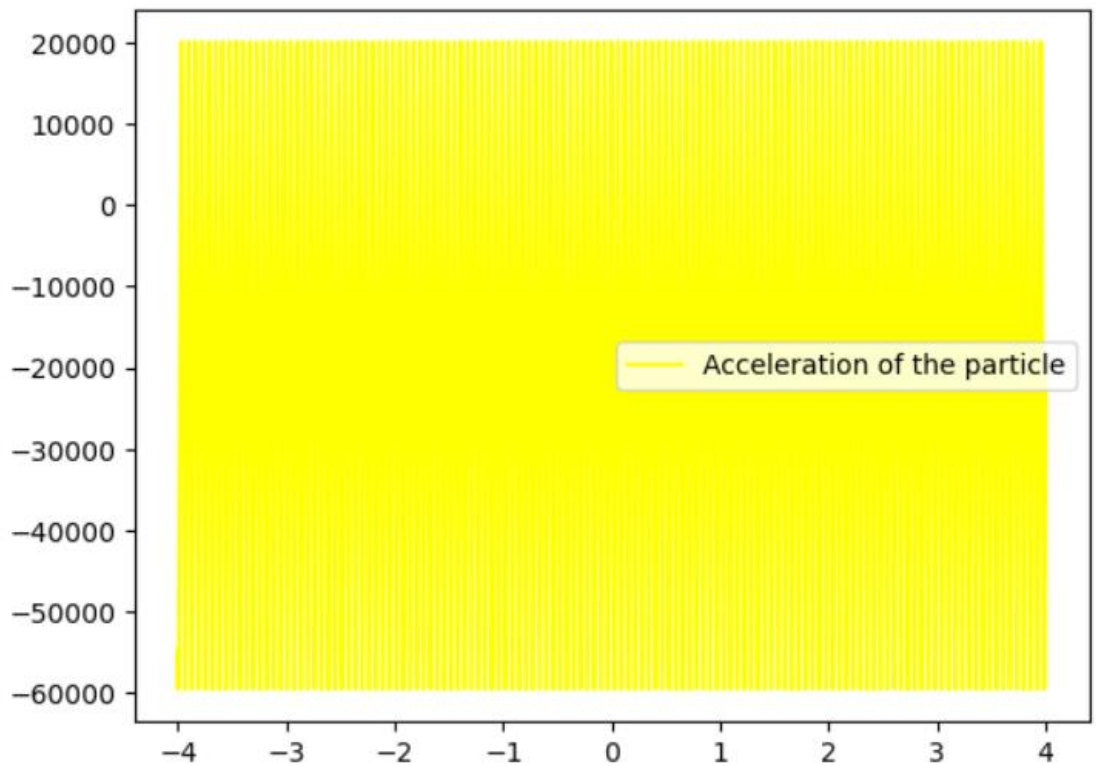
[25]:
```
import matplotlib.pyplot as plt
import numpy as np

x=-1
A =0.02
h =0.001
w=100
s= (np.pi)/6
f= A*np.cos((w*x)+s)
order1 = 1
order2 = 2

def central_diff(f,x,h,order):

    fhp =A*np.cos((w*(x+h))+s)
    fhn =A*np.cos((w*(x-h))+s)

    if order == 2:
        f1 = (fhn - (2*f) +fhp ) /(h**2)
    elif order == 1:
        f1 = (fhp- fhn) /(2*h)
    else:
        print('enter valid order(1 or 2)')
    return f1


f1 = central_diff(f,x,h,order1)
#print velosity of t = x point)
print ('Speed of the particle in t =',x,' time is =',f1)
```

```
f1 = central_diff(f,x,h,order1)
#print velosity of t = x point)
print ('Speed of the particle in t =',x,' time is =',f1)

f1 = central_diff(f,x,h,order2)
#print velosity of t = x point)
print ('Acceleration of the particle in t =',x,' time is =',f1)

t=np.linspace(-4,4,8000)
X= A*np.cos((w*t)+s)
plt.plot(t, X, color="green", label="Simple Harmonic motion")
plt.legend()
plt.show()

X1= central_diff(f,t,h,order1)
plt.plot(t, X1, color="blue", label="Speed of the particle")
plt.legend()
plt.show()

X1= central_diff(f,t,h,order2)
plt.plot(t, X1, color="yellow", label="Acceleration of the particle")
plt.legend()
plt.show()
```

```
Speed of the particle in t = -1  time is = -1.7364723892102267
Acceleration of the particle in t = -1  time is = -98.6392053480157
```







3. Integration: Trapezoidal Rule

i.

a) trapezoidal rule for the definite integral:

int $= \frac{h}{2}[\, f(a) + 2 \ \sum_{i=1}^{n-1} f(xi) + f(b)]$

Where h=(b-a)/n is the width of each sub interval and xi=a+ih for i=1,2,…,n−1

b) advantages of using the trapezoidal rule for numerical integration:

- **Simple to understand and implement** as it uses a linear approximation (trapezoid) between points and requires fewer calculations than more complex rules.
- **Works for most continuous functions**.Especially suitable for smooth and slowly varying functions.
- **Adaptable to irregular data** as it can be modified for non-uniform spacing if necessary.
- **Numerically stable for well-behaved functions** . It does not suffer from oscillatory behavior like some higher-order methods

c) For a single interval, the truncation error is: TEi = - (h^3/12)$f''(xi)$ + O(h^4)

For the complete integral over n intervals: TE=$\sum_{i=1}^{n}$ TEi

TE= -n(h^3/12)f''(c)

TE= -(b-a)(h^2/12)f''(c)

where c is some point in [a,b].

SO The trapezoidal rule has second-order accuracy O(h^2). This means the error decreases quadratically as the step size h becomes smaller.

d) Limitations of the trapezoidal rule :

- Not good for functions with sharp changes or high curvature.It requires many subdivisions for accurate results with curved functions.
- Cannot handle discontinuities well so thet errors may increase drastically near discontinuous points.
- Poor performance with rapidly oscillating functions.
- Endpoint values heavily influence overall accuracy therefore errors at boundaries can dominate the result.

e) We can split the integral at discontinuity points and apply trapezoidal rule seperately to each continuous segment and finally add the results from all segments.
If f(x) has a discontinuity at c in [a,b] then we have to apply the trapezoidal rule on [a,c) and (c,b].

f) The trapezoidal rule can be used for high-frequency functions, but its accuracy strongly depends on a smaller step size (h) to resolve the rapid changes.Therefore we can increase the number of subintervals n to capture oscillations.

ii.

```python
import numpy as np

def trapz(f, a, b, n):
    if n <= 0:
        print("n must be a positive integer.")

    h = (b - a) / n

    x = np.linspace(a, b, n + 1)

    y = f(x)

    integral = (y[0] + y[-1]) / 2.0

    integral += np.sum(y[1:-1])

    integral *= h

    return integral

def my_f(x_val):
    return 2 * np.pi * (x_val**2.5)

a = 0
b = 2
n = 200

calc = trapz(my_f, a, b, n)

actual = (2 * np.pi * (b**3.5)) / 3.5
```


```python
    integral *= h

    return integral

def my_f(x_val):
    return 2 * np.pi * (x_val**2.5)

a = 0
b = 2
n = 200

calc = trapz(my_f, a, b, n)

actual = (2 * np.pi * (b**3.5)) / 3.5
```


```python
print("Calculated integral:",calc)
print("Actual integral: ",actual)

err = calc - actual
print("Integration error:",err)
```

```
Calculated integral: 20.310692248490163
Actual integral:  20.310322003009674
Integration error: 0.0003702454804894728
```

iii.

[ ]:
```
# Q iii)
```

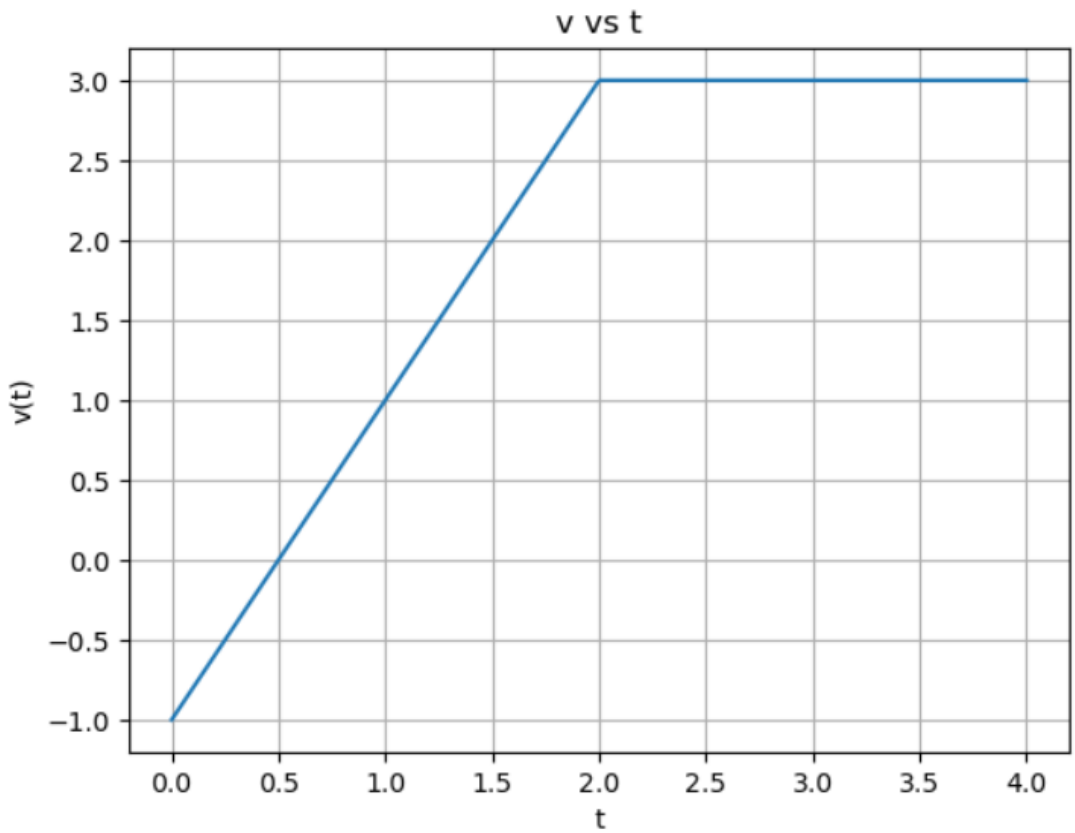
[1]:
```
import numpy as np

def f(x):
    return 2 * np.pi * x**2 * np.sqrt(x)

def trap(f, a, b, h):
    x = np.arange(a, b + h, h)
    y = f(x)
    return h * (np.sum(y) - 0.5 * (y[0] + y[-1]))

r = 2
exact = 2 * np.pi * r**3 / 3
hs = [0.5, 0.1, 0.05, 0.01, 0.001]

for h in hs:
    approx = trap(f, 0, r, h)
    err = abs((approx - exact) / exact) * 100
    print(h, approx, exact, err)
```

```
0.5 21.2399302507689 16.755160819145562 26.76649588763566
0.1 20.347361792981328 16.755160819145562 21.439370308705588
0.05 20.31957943256527 16.755160819145562 21.27355655904398
0.01 20.310692248490163 16.755160819145562 21.22051508620445
0.001 20.310325705413806 16.755160819145562 21.218327443362263
```

iv)

]:
```python
# Q3 iv)
```

]:
```python
import numpy as np
import matplotlib.pyplot as plt

def v(t):
    return np.where(t < 2, 2 * t - 1, 3)

def trap(f, a, b, h):
    x = np.arange(a, b + h, h)
    y = f(x)
    return h * (np.sum(y) - 0.5 * (y[0] + y[-1]))

t = np.linspace(0, 4, 400)
plt.plot(t, v(t))
plt.xlabel("t")
plt.ylabel("v(t)")
plt.title("v vs t")
plt.grid(True)
plt.show()

s = 8
hs = [0.5, 0.1, 0.05, 0.01, 0.001]

for h in hs:
    a = trap(v, 0, 4, h)
    e = abs((a - s) / s) * 100
    print(f"h = {h}, s = {a:.6f}, exact = {s:.6f}, error = {e:.6f}")
```




```
h = 0.5, s = 8.000000, exact = 8.000000, error = 0.000000
h = 0.1, s = 8.000000, exact = 8.000000, error = 0.000000
h = 0.05, s = 8.000000, exact = 8.000000, error = 0.000000
h = 0.01, s = 8.000000, exact = 8.000000, error = 0.000000
h = 0.001, s = 8.003000, exact = 8.000000, error = 0.037500
```

4. Integration: Simpson's Rule

i.

a) Simpson's 1/3 Rule (with n even subintervals where n must be even)

$$_a\int^b f(x)\,dx = (h/3)\,[f(x_0) + 4\sum_{odd\ i=1}^{n-1} f(xi) + 2\sum_{even\ i=1}^{n-2} f(xi) + f(x_n)]$$

where $h = (b-a)/n$ and $x_o = a$, $x_n = b$

b) Advantages of Simpson's 1/3 Rule over Trapezoidal Rule:

- Simpson's rule uses parabolic approximation, leading to more accurate results for smooth functions.
- Gives exact results for polynomials up to degree 3, while trapezoidal rule is only exact up to degree 1.

- Since Simpson's rule uses parabolic approximation instead of linear,it is more accurate for functions with curvature.
- Faster Convergence.

c) The order of accuracy of Simpson's 1/3 Rule is O(h^4), where h is the width of each subinterval. This means the error decreases rapidly as the number of intervals increases.

d) Limitations of the Simpson's 1/3 Rule:
   - Requires an even number of intervals
   - Not well-suited for functions with discontinuities or non-smooth behavior.
   - May perform poorly if the function has high-frequency oscillations within subintervals.
   - More function evaluations per interval compared to simpler methods.

e) We can divide the integration interval at each discontinuity point and apply Simpson's rule seperately to each continuous subinterval and finally add the results from all subintervals.
   If f(x) has a discontinuity at c in [a,b] then we have to apply the Simpson's rule on [a,c) and (c,b].

   $$\int_a^b f(x)\,dx = \int_a^c f(x)\,dx + \int_c^b f(x)\,dx$$

f) When using Simpson's 1/3 Rule for Functions with high frequency or rapid changes , we have to use a very small step size (h) by increasing the number of subintervals.Also we can combine results from different step sizes to improve accuracy.

ii.

[1]:
```
# Q4 ii)
```

[2]:
```
import numpy as np

start = 0
end = 2
points = 2000

def simpson_rule(start, end, points):
    x = np.linspace(start, end, points)
    y = 2 * np.pi * x**2.5
    h = (end - start) / (points - 1)
    integral = (h/3) * (y[0] + 2 * sum(y[2:-1:2]) + 4 * sum(y[1::2]) + y[-1])
    return integral

calculated = simpson_rule(start, end, points)
actual = (2 * np.pi * end**3.5) / 3.5

print("Actual value:", actual)
print("Calculated value:", calculated)
print("Error:", actual - calculated)
```

```
Actual value: 20.310322003009674
Calculated value: 20.345890259147957
Error: -0.035568256138283516
```